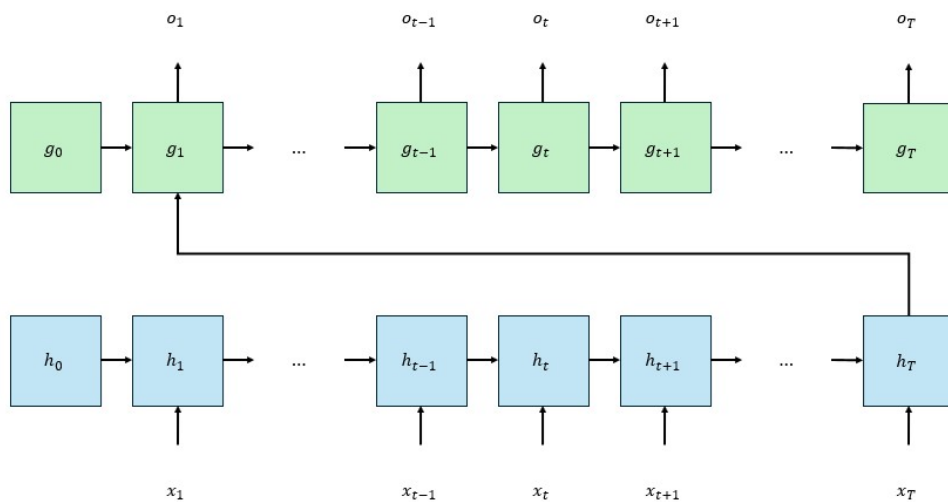
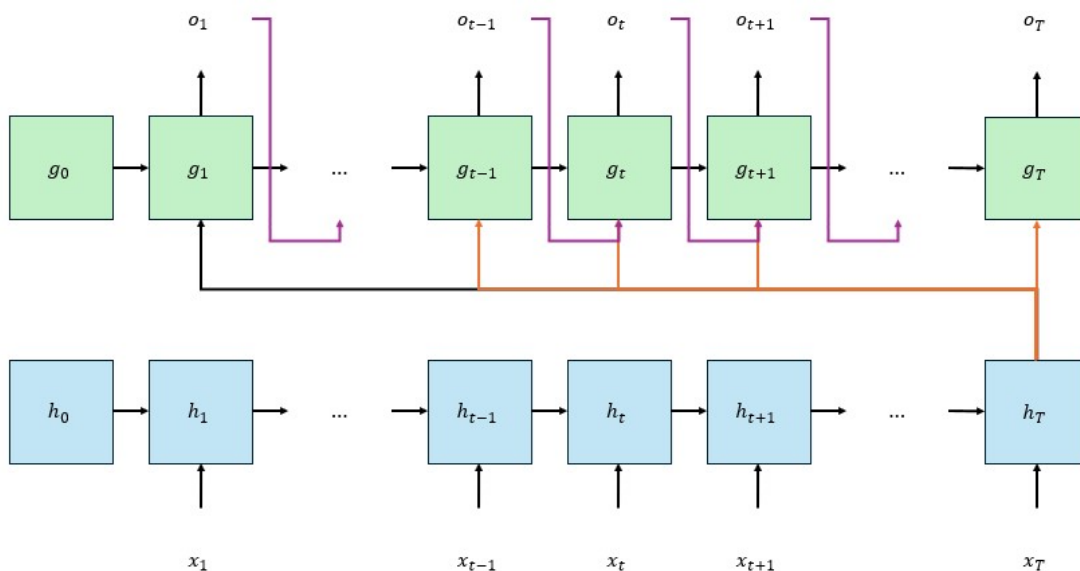


Attention

March 30, 2026 8:39 AM



1. Use o_{t-1} as input to compute state g_t .
2. Use h_T as input at all times.



$$g_t = f(g_{t-1}, o_{t-1}, h_T)$$

Still cannot model long-term dependencies in encoder.

Context vector contains a combination of hidden states over all times in encoder.

$$C_i = \sum_j \alpha_{i,j} h_j$$

$$g_i = f(g_{i-1}, o_{i-1}, C_i)$$

i indexes times in decoder
 j indexes time in encoder

Coefficients $\alpha_{i,j}$ tell us how much attention to pay to each h_j .

$$\alpha_{i,j} = \text{softmax}(\text{score}(g_{i-1}, h_j))$$

softmax ensure $\sum_j \alpha_{i,j} = 1$

$$\text{score}(g_{i-1}, h_j) = \begin{cases} \uparrow & \text{if inputs near } j \text{ match with outputs near } i \\ \downarrow & \text{if inputs near } j \text{ do not match with outputs near } i \end{cases}$$

Additive Scoring

$$\text{score}(g_{i-1}, h_j) = w_a \tanh(W_g g_{i-1} + W_h h_j)$$

w_a, W_h, W_g are learnable parameters

Multiplicative Scoring

$$\text{score}(g_{i-1}, h_j) = g_{i-1}^T W h_j$$

W is a learnable parameter.

Dot-Product Scoring

$$\text{score}(g_{i-1}, h_j) = g_{i-1}^T h_j$$

Scaled Dot-Product Scoring

$$\text{score}(g_{i-1}, h_j) = \frac{g_{i-1}^T h_j}{\sqrt{\text{len}(h_j)}}$$

Generalized Attention

Computed context vector c_i as a weighted sum of ~~encodings~~ ^{values}, where the weights are determined by how compatible the ~~current decoder state~~ ^{query} is with ~~encodings~~ ^{key}.

by how compatible ...
with each ~~encoding~~ ^{key}.

$$c_i = \sum_j \text{softmax}(\text{score}(q_i, h_j)) h_j$$

$$\text{attention}(q, k, v) = \sum_j \text{softmax}(\text{score}(q_i, k_j)) v_j$$

Example

Hospital database: store patient data by
birthdate.

{ 2021-10-31: 72cm,
2020-02-03: 91cm,
1980-12-21: 188cm
}

Suppose we want to know height of
young patients.

$$\text{score}(q, k_1) = 0.92$$

$$\text{score}(q, k_2) = 0.90$$

$$\text{score}(q, k_3) = 0.30$$

$$\sum_j \text{softmax}(\text{score}(q, k_j)) v_j$$

$$= 0.397 \cdot 72 \text{ cm} + 0.389 \cdot 91 \text{ cm} + 0.214 \cdot 188 \text{ cm}$$

$$= 104.2 \text{ cm}$$

Example

$$q = [2] \quad , \quad k = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

$$v = \begin{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} , \begin{bmatrix} 0 \\ 1 \end{bmatrix} , \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{bmatrix}$$

$$\text{attention}(q, k, v) =$$

Use the dot-product scoring function.

$$\text{score}(q, k_1) = 2$$

$$\text{score}(q, k_2) = 0$$

$$\text{score}(q, k_3) = -2$$

$$\text{softmax}(\text{score}(q, k_1)) = 0.867$$

$$\text{softmax}(\text{score}(q, k_2)) = 0.117$$

$$\text{softmax}(\text{score}(q, k_3)) = 0.016$$

$$\begin{aligned} \text{attention}(q, k, v) &= 0.867 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 0.117 \begin{bmatrix} 0 \\ 1 \end{bmatrix} + 0.016 \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 0.883 \\ 0.984 \end{bmatrix} \end{aligned}$$

Types of Attention

1. 11. 12.

types ...

Bahdanau Attention

$q_i = g_{i-1}$ (the prior state in RNN decoder)

$k_j = v_j = h_j$ (the hidden states in RNN encoder).

Self-Attention

Imagine input sequence $x_1, \dots, x_T$

Compute q, k, v as linear functions of input

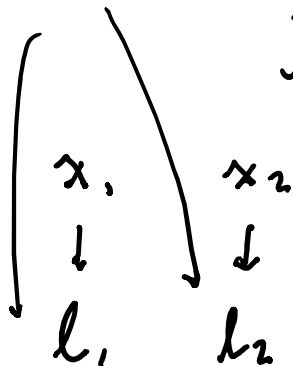
$$q_i = W_q x_i$$

$$k_i = W_k x_i$$

$$v_i = W_v x_i$$

attention(q_i, k, v)

$$c_i = \sum_j \text{softmax}(\text{score}(q_i, k_j)) v_j$$



↓ h_1 h_2

